

AURALIS

A Smart Cane That Remembers

Research, Ideation and Design Plan · Artificial & Neural Networks Capstone

1. What we are building

A smart white cane for visually impaired people, with hardware and a mobile web app working together. It must tell the user what obstacle is in front through multilingual voice (phone speaker or Bluetooth earphones), and it must **remember locations and hazards**. It must contain real neural networks (course requirement) and be novel enough for a research paper. Timeline: 2-3 days.

One-line claim: AURALIS turns every hazard it detects into a persistent, time-decaying memory and uses that memory to warn early and choose safer routes. Existing smart canes only react to what is in front of them right now.

2. Research: what already exists

Obstacle detection with sensors, cameras and YOLO is already well studied, so it is our **baseline**, not our novelty.

Existing work	What it does	What it lacks
WeWALK Smart Cane 2 (commercial, CES 2025)	Chest-height ultrasonic alerts, app, turn-by-turn navigation, AI assistant	No learned hazard memory; expensive
Stanford Augmented Cane (2021, \$400)	LiDAR, GPS, IMU, SLAM, motorised wheel steers user; +20% walking speed	Heavy, complex; no long-term hazard memory
Low-cost YOLO smart cane (MDPI Information, 2025)	Dual RGB cameras + ultrasonic, YOLOv5x/YOLOv8n, surface transition detection	Reactive only
CV + depth smart cane (arXiv 2025)	Camera + depth sensor real-time detection	Reactive only
LiDAR + RGB-D cane (PMC)	2D LiDAR, RGB-D, GPS, IMU, SLAM, improved YOLOv5	Costly sensors; no risk memory
Ultrasonic cane prototype (Springer 2025) , S-Cane	Ultrasonic distance + audio / Bluetooth app alerts	No vision, no memory
AI smart cane survey (2025) , navigation survey	Reviews of the field; crowdsourced hazard reporting mentioned as a direction	-

The gap: almost every system is reactive. It warns about what is in front now and forgets it immediately. None learns the user's routes with hazards that strengthen on repeated sightings and fade over time, and then uses that to route around danger. That gap is our research contribution.

3. Chosen features (1 + 2 + 3)

#	Feature	How it works	Why it is novel
1	Hazard memory that fades	Each confirmed static hazard is saved with GPS, time, class, confidence. Seen again -> stronger. Not seen -> decays (half-life ~7 days). User can say 'hazard gone'.	Warns ~15-25 m before the sensors can see it, vs ~1-2 m for reactive canes
2	Self-learning safest route	Walks are recorded as a path graph (no map data needed). Saved places by voice ('library'). A* route cost = length x (1 + lambda x risk).	Prefers a slightly longer, safer path; works on campuses that maps don't cover
3	Neural sensor fusion	MLP combines camera class/confidence/box size/position, distances, approach speed and memory risk into one alert level.	Learned fusion compared against if-else rules in the paper

#	Feature	How it works	Why it is novel
+	Multilingual voice	Phone text-to-speech: English, Hindi, Telugu, Tamil (extendable), via speaker or Bluetooth earphones.	Accessibility for Indian users

Risk of a remembered hazard: $\text{Risk} = \text{Severity} \times (1 - e^{-\text{count}/2}) \times 0.5^{\text{age} / 7 \text{ days}} \times \text{Confidence}$

Two neural networks (course requirement): (1) a CNN detector (YOLO) fine-tuned on our obstacle classes, including potholes and drains; (2) a fusion MLP that outputs the alert level.

4. What counts as an obstacle

Definition: an obstacle is anything inside the user's **walking corridor** that could cause contact, a trip or a fall. The corridor is about 1 m wide, from the ground up to head height (about 2 m), and up to 4 m ahead. Objects outside the corridor are tracked silently and not announced, so the cane does not talk nonstop.

#	Category	Examples	Detected by	Saved to memory?
1	Drop / pit	pothole, open drain / manhole, step down, stairs down, curb edge	Down sensor (depth jump) + camera	Yes (highest severity)
2	Raised ground	step up, stairs up, speed breaker, stones / debris	Down sensor + camera	Yes
3	Body-level static	pole, tree, wall, parked bike / car, bench, dustbin, barrier	Front sensor + camera	Yes
4	Head-level	low branches, signboards, shop shutters, truck back-end	Head-level sensor	Yes (a normal cane misses these)
5	Moving	person, bicycle, motorbike, car, auto-rickshaw, bus, dog, cow	Camera + approach speed	No (announced only)
6	Zone / context (optional)	construction area, wet floor, crowd	Camera	Yes, as route-level risk

Detector labels. Already in pre-trained (COCO) models: person, bicycle, motorbike, car, bus, truck, dog, cow, bench, chair. **We must train:** pothole, open drain, stairs, curb, pole, auto-rickshaw, using public pothole/stairs datasets plus 100-200 of our own photos.

Alert levels

Level	When	Vibration	Voice example
Silent	Outside corridor, or > 4 m	none	-
Caution	1.5-4 m, or a remembered hazard ahead	1 short pulse	"Pole ahead, 3 metres"
Warning	0.75-1.5 m, or something approaching fast	2 pulses	"Bike on your left, 1 metre"
Stop	< 0.75 m, or a drop / pit at any distance	strong repeated pulses	"Stop. Pit ahead"

The same alert is not repeated within a few seconds (cooldown), and a higher level always interrupts a lower one.

5. Hardware

Main: Option B (Raspberry Pi 5). Fallback: Option A (ESP32 + phone) if a Pi is not available. Both use the same mobile web app.

	B: Raspberry Pi 5 (main)	A: ESP32 + phone (fallback)
Camera	Pi Camera Module 3 on the cane handle	Phone rear camera (clip on cane or chest)
Object detection runs on	Pi (YOLO, ONNX / NCNN)	Phone browser (TensorFlow.js)
Distance sensors	3x VL53L1X ToF (front, down 35-45 deg, head-level) via TCA9548A	2-3x ultrasonic (front, down, head-level)
Vibration	DRV2605L driver + 2 coin motors (left / right)	Transistor + 1-2 coin motors
Link to phone	Wi-Fi: phone opens the web app served by the Pi	Bluetooth Low Energy (Web Bluetooth)
Phone does	Voice, earphones, GPS, memory, routing	Same + camera + detection
Approx. cost	Rs. 12,000-15,000	Rs. 1,500-2,000

Bill of materials (Option B)

Part	Qty	Purpose
Raspberry Pi 5 (4/8 GB) + 64 GB microSD + active cooler	1	Main AI processor
Raspberry Pi Camera Module 3	1	Forward vision
VL53L1X ToF sensor + TCA9548A I2C multiplexer	3 + 1	Front, downward, head-level distance
MPU6050 / BNO055 IMU	1	Cane tilt, to correct the down sensor
DRV2605L haptic driver + coin vibration motors	1 + 2	Left / right / urgent patterns
Push buttons	2-3	Where am I / save place / SOS
USB-C power bank (5V 3A, 10,000-20,000 mAh)	1	Portable power
PVC pipe or white cane, 3D-printed / PVC mounts, wires	-	Body
Bluetooth earphones (user's own)	1	Voice output via phone

Option A parts: ESP32 DevKit, 2-3 HC-SR04 (with 1k/2k voltage divider on ECHO) or JSN-SR04T, coin motor + 2N2222 + 1N4007 diode, button, power bank.

6. Software

Layer	Choice
App	Mobile web app (Chrome on Android), installable as a PWA; works offline after first load
Detection	YOLOv8n / YOLO11n fine-tuned on Google Colab; exported for Pi (B) or TensorFlow.js (A)
Fusion	Small MLP (8 inputs -> 16 -> 16 -> 4 alert levels), trained in Python, runs on phone
Memory	On the phone (IndexedDB / localStorage): hazards, places, path graph
Routing	Risk-aware A* on the self-built path graph
Voice	Phone text-to-speech + speech recognition for naming places

Data flow: sense (camera + distances + GPS) -> detect (YOLO) -> fuse (MLP) -> alert (voice + vibration) -> remember (hazard map) -> route (A*) -> learn (repeat sightings / decay).

7. 3-day build plan

Day	Hardware	Software / AI	Paper
1	Get parts; confirm Pi or ESP32. Wire sensors + motors, mount on cane, test distances at 0.5/1/2/3 m	Collect 100-200 obstacle photos; label in Roboflow / CVAT	Related work section
2	Integrate camera; field walks on 2 campus routes	Fine-tune YOLO on Colab; build web app: voice, memory, places, routing	Method section
3	Final packaging; demo video	Train + evaluate fusion MLP; run experiments	Results, discussion, abstract

8. Experiments for the paper

#	Experiment	Metric
E1	Sensor accuracy at 0.5 / 1 / 2 / 3 m	Mean absolute error
E2	YOLO on our obstacle classes	Precision, recall, mAP@0.5
E3	Fusion MLP vs if-else rules	Accuracy, urgent-class recall, confusion matrix
E4	Early warning: walk a route 5 times with a fixed obstacle	Warning distance: reactive vs memory

#	Experiment	Metric
E5	Shortest vs safest route for $\lambda = 0, 1, 2, 4, 8$	Path length vs accumulated risk
E6	Alert latency (obstacle appears -> voice starts)	Milliseconds
E7	Multilingual voice clarity, 5 listeners	Mean rating 1-5

9. Honest limitations

- Phone GPS is about +/-5-10 m: good for 'hazard ahead' memory, not for exact sidewalk steering.
- Custom classes (pothole, drain) need training data; accuracy depends on how many photos we collect.
- Web Bluetooth works in Chrome on Android, not iPhone Safari (only matters for Option A).
- Field testing with blindfolded sighted volunteers, not blind users, within the 3-day scope.

10. Open decisions

- Is a Raspberry Pi 5 (or Pi 4) available? Which parts are already in hand?
- Languages: English, Hindi, Telugu, Tamil, or others?
- Rework the existing draft code in the auralis- repo, or start fresh?